راهنمای بلوک‌به‌بلوک کد رگرسیون ساده

توضیح دو برنامه‌ی `simple_regression_plot.R` و `simple_regression_plot.py` (بدون بخش گرافیکی GUI)

1. این کدها چه کاری انجام می‌دهند

هر دو برنامه یک کار را انجام می‌دهند: مقادیر x و y را از شما می‌گیرند، رگرسیون خطی ساده را حساب می‌کنند و نقاط داده را همراه با خط رگرسیون رسم می‌کنند. ساختار هر دو یکسان و در سه مرحله است:

- **دریافت داده:** خواندن x و y و اطمینان از معتبر بودن آن‌ها
- **محاسبه:** برازش خط و چاپ نتایج
- **نمودار:** رسم نقاط و خط رگرسیون

2. ایده‌ی ریاضی در یک نگاه

رگرسیون خطی ساده خطی به شکل `y = a + b*x` پیدا می‌کند که به نقاط داده «نزدیک‌ترین» باشد. نزدیک‌ترین یعنی **کمترین مربعات**: برای هر نقطه فاصله‌ی عمودی آن تا خط (باقیمانده) محاسبه می‌شود و خطی انتخاب می‌شود که مجموع مربعات این فاصله‌ها کمینه باشد. جواب این مسئله فرمول بسته دارد:

```
b = sum((x - mean(x)) * (y - mean(y))) / sum((x - mean(x))^2)
a = mean(y) - b * mean(x)
```

کمیت	معنی
`b` (شیب، Slope)	به ازای هر یک واحد افزایش x، مقدار y به‌طور میانگین چقدر تغییر می‌کند.
`a` (عرض از مبدأ، Intercept)	مقدار پیش‌بینی‌شده‌ی y وقتی x برابر صفر است.
R^2	نسبتی از تغییرات y که خط توضیح می‌دهد؛ عددی بین 0 و 1.
خطای معیار (Std. Error)	عدم قطعیت برآورد یک ضریب؛ هرچه کوچک‌تر باشد برآورد دقیق‌تر است.
p-value	نتیجه‌ی آزمون «شیب برابر صفر است»؛ مقدار کوچک (معمولاً کمتر از 0.05) یعنی شواهد کافی برای وجود رابطه‌ی خطی هست.

3. کد R (simple_regression_plot.R)

اجرا: داخل R یا RStudio با دستور `source("simple_regression_plot.R")` . با `Rscript` اجرا نکنید، چون `readline` فقط در جلسه‌ی تعاملی از کاربر ورودی می‌گیرد.

بلوک 1: تابع خواندن عددها خطوط 8 تا 11

```
read_numbers <- function(prompt) {
  text <- gsub(",", " ", readline(prompt))
  scan(text = text, quiet = TRUE)
}
```

- `function(prompt)` : تابعی به نام `read_numbers` با یک ورودی به نام `prompt` (پیامی که به کاربر نشان داده می‌شود).
- `readline(prompt)` : پیام را نمایش می‌دهد و آنچه کاربر تایپ می‌کند را به شکل یک رشته‌ی متنی برمی‌گرداند.
- `gsub(",", " ", ...)` : همه‌ی ویرگول‌ها را به فاصله تبدیل می‌کند؛ بنابراین هم `1 2 3` و هم `1, 2, 3` پذیرفته می‌شود.
- `scan(text = text, quiet = TRUE)` : رشته را به بردار عددی تبدیل می‌کند. `quiet = TRUE` پیام «Read N items» را حذف می‌کند. اگر بخشی از متن عدد نباشد، خود `scan` خطا می‌دهد.
- آخرین عبارت تابع (`scan`) مقدار برگشتی است؛ در R نیازی به `return` نیست.

بلوک 2: گرفتن x و y خطوط 13 و 14

```
x <- read_numbers("Enter the x values: ")
y <- read_numbers("Enter the y values: ")
```

- تابع دو بار صدا زده می‌شود و هر بار یک بردار عددی در `x` و `y` ذخیره می‌شود.
- عملگر `<-` یعنی «مقدار سمت راست را در متغیر سمت چپ بگذار».

بلوک 3: بررسی ورودی خطوط 16 و 17

```
if (length(x) != length(y)) stop("x and y must have the same number of values.")
if (length(x) < 3) stop("Enter at least 3 pairs of values.")
```

- `length(x)` تعداد مقدارهای بردار است. اگر تعداد x و y برابر نباشد، جفت‌سازی نقاط ممکن نیست.
- `stop("...")` اجرای برنامه را متوقف می‌کند و پیام خطا را چاپ می‌کند.
- چرا حداقل 3 جفت؟ با 2 نقطه، خط دقیقاً از هر دو می‌گذرد و درجه‌ی آزادی باقیمانده (n - 2) صفر می‌شود؛ در نتیجه خطای معیار و p-value قابل محاسبه نیستند.

بلوک 4: محاسبه‌ی رگرسیون خطوط 20 و 21

```
model <- lm(y ~ x)
print(summary(model))
```

- `lm` مخفف linear model است. فرمول `y ~ x` یعنی «y را بر حسب x مدل کن»؛ عرض از مبدأ به‌طور خودکار اضافه می‌شود. نتیجه یک شیء مدل است که ضرایب، باقیمانده‌ها و مقدارهای برازش‌شده را نگه می‌دارد.
- `summary(model)` آماره‌های تکمیلی را حساب می‌کند: خطای معیار، t، p-value، R^2 و F.
- `print(...)` لازم است، چون هنگام اجرای فایل با `source()` نتیجه‌ی عبارت‌ها به‌طور خودکار چاپ نمی‌شود.

بلوک 5: رسم نمودار خطوط 24 تا 29

```
equation <- sprintf("y = %.3f + %.3f x   (R-squared = %.3f)",
                    coef(model)[1], coef(model)[2], summary(model)$r.squared)

plot(x, y, pch = 19, col = "steelblue", xlab = "x", ylab = "y",
     main = equation)
abline(model, col = "red", lwd = 2)
```

- `sprintf` متن را از روی یک قالب می‌سازد. `%.3f` یعنی عدد اعشاری با 3 رقم اعشار و سه مقدار بعدی به ترتیب جایگزین می‌شوند.
- `coef(model)[1]` عرض از مبدأ، `coef(model)[2]` شیب و `summary(model)$r.squared` مقدار R^2 است.

- `plot(x, y, ...)` نمودار پراکندگی را می‌کشد: `pch = 19` یعنی دایره‌ی توپر، `col` رنگ نقاط، `xlab` و `ylab` برچسب محورها و `main` عنوان (همان معادله).
- `abline(model, ...)` خط رگرسیون را روی نمودار موجود می‌کشد و ضرایب را مستقیماً از مدل می‌خواند؛ `lwd` ضخامت خط است.

خواندن خروجی summary

خروجی نمونه برای داده‌ی x = 1..8 (بخش اصلی):

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.15357    0.14281   1.075    0.324
x            1.97976    0.02828  70.004 5.72e-10 ***

Residual standard error: 0.1833 on 6 degrees of freedom
Multiple R-squared:  0.9988,  Adjusted R-squared:  0.9986
F-statistic:  4901 on 1 and 6 DF,  p-value: 5.717e-10
```

بخش خروجی	معنی
`Estimate`	برآورد ضریب: در ردیف `(Intercept)` همان `a` و در ردیف `x` همان شیب `b`.
`Std. Error`	خطای معیار برآورد هر ضریب.
`t value`	نسبت Estimate به Std. Error؛ هرچه بزرگ‌تر، ضریب با اطمینان بیشتری غیرصفر است.
`Pr(>|t|)`	p-value آزمون صفر بودن همان ضریب.
`Residual standard error`	انحراف معیار باقیمانده‌ها با n - 2 درجه‌ی آزادی؛ اندازه‌ی خطای معمول پیش‌بینی در واحد y.
`Multiple R-squared`	R^2؛ `Adjusted R-squared` همان R^2 است که برای تعداد پارامترهای مدل تعدیل شده است.
`F-statistic`	آزمون کل مدل. در رگرسیون ساده p-value آن با p-value شیب یکسان است.

4. کد Python (simple_regression_plot.py)

اجرا: `python simple_regression_plot.py` پس از نصب کتابخانه‌ها با `pip install numpy scipy matplotlib`.

بلوک 1: کتابخانه‌ها خطوط 9 تا 11

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

- `numpy` (با نام کوتاه `np`): آرایه‌ها و محاسبات عددی.
- `matplotlib.pyplot` (با نام کوتاه `plt`): رسم نمودار.
- `scipy.stats`: شامل تابع `linregress` که رگرسیون خطی ساده را حساب می‌کند.
- رشته‌ی سه‌گانه‌ی ابتدای فایل (خطوط 1 تا 7) فقط توضیح فایل است و اجرا نمی‌شود.

بلوک 2: تابع خواندن عددها خطوط 14 تا 20

```
def read_numbers(prompt):
    """Reads numbers separated by spaces or commas."""
    text = input(prompt).replace(",", " ")
    try:
        return np.array([float(value) for value in text.split()])
    except ValueError:
        raise SystemExit("Please enter numbers only.")
```

- `def read_numbers(prompt)` : تعریف تابع. رشته‌ی سه‌گانه‌ی داخل آن توضیح تابع (docstring) است.
- `input(prompt)` : پیام را چاپ می‌کند و یک خط از ورودی را به شکل رشته برمی‌گرداند؛ `.replace(",", " ")` ویرگول‌ها را به فاصله تبدیل می‌کند.
- `text.split()` : رشته را در فاصله‌ها می‌شکند و فهرستی از رشته‌ها می‌دهد.
- `[float(value) for value in ...]` : ساخت فهرست با حلقه‌ی فشرده؛ هر قطعه به عدد اعشاری تبدیل می‌شود.
- `np.array(...)` : فهرست را به آرایه‌ی numpy تبدیل می‌کند تا بعداً بتوان روی کل آرایه محاسبه کرد (مثل `x.min()` و عملیات برداری).
- `try / except ValueError` : اگر `float` نتواند قطعه‌ای را تبدیل کند (مثلاً `abc`) خطای `ValueError` رخ می‌دهد. به جای نمایش خطای طولانی، `SystemExit` برنامه را با یک پیام کوتاه متوقف می‌کند.

بلوک 3: گرفتن داده و بررسی آن خطوط 24 تا 32

```
x = read_numbers("Enter the x values: ")
y = read_numbers("Enter the y values: ")

if len(x) != len(y):
    raise SystemExit("x and y must have the same number of values.")
if len(x) < 3:
    raise SystemExit("Enter at least 3 pairs of values.")
if np.all(x == x[0]):
    raise SystemExit("All x values are identical, so a regression line cannot be computed.")
```

- دو شرط اول مانند R هستند: تعداد برابر و حداقل 3 جفت.
- `x == x[0]` هر عنصر را با اولین عنصر مقایسه می‌کند و آرایه‌ای از True و False می‌دهد؛ `np.all` درست بودن همه‌ی آن‌ها را بررسی می‌کند.
- اگر همه‌ی x برابر باشند، مخرج فرمول شیب (`sum((x - mean(x))^2)`) صفر می‌شود و شیب تعریف‌نشده است.

بلوک 4: محاسبه و چاپ نتایج خطوط 35 تا 47

```
try:
    result = stats.linregress(x, y)
except ValueError as error:
    raise SystemExit(str(error))

r_squared = result.rvalue ** 2

print("\nRegression results")
print(f"  Intercept : {result.intercept:.4f}  (std. error {result.intercept_stderr:.4f})")
print(f"  Slope     : {result.slope:.4f}  (std. error {result.stderr:.4f})")
print(f"  R-squared : {r_squared:.4f}")
print(f"  p-value   : {result.pvalue:.3g}  (test of slope = 0)")
print(f"  n         : {len(x)}")
```

- `stats.linregress(x, y)` رگرسیون را حساب می‌کند و شیئی با این ویژگی‌ها برمی‌گرداند: `slope` (شیب)، `intercept` (عرض از مبدأ)، `rvalue` (ضریب همبستگی)، `pvalue` (آزمون شیب برابر صفر)، `stderr` (خطای معیار شیب) و `intercept_stderr` (خطای معیار عرض از مبدأ).
- `try / except` خطاهای احتمالی محاسبه را به یک پیام کوتاه تبدیل می‌کند.
- `result.rvalue ** 2` : در رگرسیون ساده R^2 برابر مربع ضریب همبستگی است (`**` یعنی توان).
- `f"..."` رشته‌ی قالب‌دار (f-string) است: عبارت داخل `{ }` محاسبه و با قالب بعد از `:` نوشته می‌شود. `:.4f` یعنی 4 رقم اعشار و `:.3g` یعنی 3 رقم معنادار (برای p-valueهای بسیار کوچک نماد علمی می‌دهد).
- `\n` ابتدای اولین پیام یک خط خالی ایجاد می‌کند.

بلوک 5: رسم نمودار خطوط 50 تا 60

```
line_x = np.linspace(x.min(), x.max(), 100)
line_y = result.intercept + result.slope * line_x

plt.scatter(x, y, color="steelblue", label="Data")
plt.plot(line_x, line_y, color="red", linewidth=2, label="Regression line")
plt.title(f"y = {result.intercept:.3f} + {result.slope:.3f} x   (R-squared = {r_squared:.3f})")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

- `np.linspace(x.min(), x.max(), 100)` : 100 نقطه‌ی هم‌فاصله بین کمترین و بیشترین مقدار x برای کشیدن خط.
- `line_y = intercept + slope * line_x` : محاسبه‌ی برداری؛ عملیات روی همه‌ی عناصر آرایه همزمان انجام می‌شود و نیازی به حلقه نیست.
- `plt.scatter` نقاط داده و `plt.plot` خط را می‌کشد. `label` نامی است که در راهنمای نمودار (legend) نمایش داده می‌شود.
- `plt.title` ، `plt.xlabel` و `plt.ylabel` عنوان و برچسب محورها را می‌گذارند؛ `plt.legend()` راهنما را نشان می‌دهد.
- `plt.grid(alpha=0.3)` شبکه‌ی کم‌رنگ می‌کشد (`alpha` شفافیت است).
- `plt.show()` پنجره‌ی نمودار را باز می‌کند و تا بسته شدن آن، برنامه منتظر می‌ماند.

5. مقایسه‌ی دو نسخه

کار	R	Python
گرفتن ورودی	`readline()`	`input()`
تبدیل متن به عدد	`scan(text = ...)`	`float(...)` و `np.array(...)`
توقف با پیام خطا	`stop("...")`	`raise SystemExit("...")`
برازش خط	`lm(y ~ x)`	`stats.linregress(x, y)`
چاپ نتایج	`print(summary(model))`	`print(f"...")` برای هر مقدار
کشیدن خط رگرسیون	`abline(model)`	`plt.plot(line_x, line_y)`

کار	R	Python
نمایش نمودار	خودکار با `plot()`	با `plt.show()`
آماره‌های اضافه	R^2 تعدیل‌شده و F در خروجی `summary`	فقط آماره‌های رگرسیون ساده (شیب، عرض از مبدأ، R^2، p-value)

6. مثال عددی: بررسی دستی نتیجه

با داده‌ی x = 1, 2, ..., 8 و y = 2.1, 4.3, 5.9, 8.2, 9.8, 12.1, 14.2, 15.9 فرمول بخش 2 را مرحله‌به‌مرحله حساب می‌کنیم:

کمیت	مقدار
میانگین x: `mean(x)`	`4.5`
میانگین y: `mean(y)`	`9.0625`
`Sxx = sum((x - mean(x))^2)`	`42`
`Sxy = sum((x - mean(x)) * (y - mean(y)))`	`83.15`
`Syy = sum((y - mean(y))^2)`	`164.81875`
شیب: `b = Sxy / Sxx`	`83.15 / 42 = 1.97976`
عرض از مبدأ: `a = mean(y) - b * mean(x)`	`9.0625 - 1.97976 * 4.5 = 0.1536`
R^2: `Sxy^2 / (Sxx * Syy)`	`83.15^2 / (42 * 164.81875) = 0.99878`

هر دو برنامه‌ی R و Python همین مقدارها را چاپ می‌کنند: شیب 1.9798، عرض از مبدأ 0.1536 و R^2 برابر 0.9988؛ یعنی هر 1 واحد افزایش x حدود 1.98 واحد y را بالا می‌برد.

7. نکات تفسیر و محدودیت‌ها

- R^2 بالا فقط یعنی خط با داده سازگار است و رابطه‌ی علت و معلولی را ثابت نمی‌کند.
- پیش‌بینی خارج از بازه‌ی x مشاهده‌شده (extrapolation) قابل‌اعتماد نیست.
- یک نقطه‌ی پرت می‌تواند شیب را به شدت تغییر دهد؛ همیشه نمودار را نگاه کنید.
- p-value و فاصله‌های اطمینان به فرض‌های مدل وابسته‌اند (رابطه‌ی خطی، استقلال و واریانس ثابت خطاها و برای نمونه‌های کوچک نرمال بودن تقریبی باقیمانده‌ها)؛ بررسی آن‌ها در `regression_tool.R` انجام می‌شود.
- این دو برنامه فقط یک متغیر مستقل دارند. اگر شیب منفی باشد، عنوان نمودار به شکل `y = -2.000 + ...` نوشته می‌شود؛ این فقط ظاهر عنوان است (در نسخه‌های گرافیکی اصلاح شده).
- اگر همه‌ی x یکسان باشند، نسخه‌ی Python پیام خطا می‌دهد؛ ولی در نسخه‌ی R تابع `lm` برای شیب `NA` برمی‌گرداند و `abline` خطای «'a' and 'b' must be finite» می‌دهد.